

BankForge — MCP Server Ecosystem for Digital Banking — Capstone Assignment

Capstone Team

Version 1.3, 2026-09-30: capstone assignment specification — everything described below must be
built from scratch; no starter kit is provided

Table of Contents

Week 5 Capstone — Model Context Protocol: Build, Compose & Deploy

Assignment specification — build all servers, tools, resources, and prompts from scratch

Field	Detail
Domain	Digital banking — NeoBank India (fictional)
Servers to build	3 (<code>accounts_server</code> , <code>products_server</code> , <code>compliance_comms_server</code>)
MCP primitives required	Tools (14 total — see §3.1), Resources (4 — see §7), Prompts (2 — see §7)
Deliverables	Working Tool/Resource/Prompt logic (with tests actually executed), architecture document, local demo script covering the Showcase Scenario and all Stress Tests
Suggested prerequisite skill level	Working knowledge of MCP primitives (Tools, Resources, Prompts) and intermediate Python

1. Project Overview

You will build BankForge: a subset of NeoBank India's (fictional) backend systems, exposed as three independently deployable MCP servers, each backed by a local SQLite database. Every tool call must be traced at DEBUG level with structured JSON logging, and you must implement a dedicated `guardrails.py` module covering scoped field visibility, PII-safe logging, input sanitisation, and deterministic compliance rules.

This document is a self-contained assignment brief describing everything you are required to build — nothing here is provided as a starter kit. If your instructor has issued a separate written specification, resolve any conflicts in that document's favor.

2. Getting Started (Local Setup)

Everything in this project runs as plain local Python processes — no Docker or other container runtime is required or expected.

2.1 Prerequisites

- Python 3.11+
- `pip`

2.2 Setup

```
# 1. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 2. Install your dependencies (define these in your own requirements.txt)
pip install -r requirements.txt
pip install pytest                # needed to run the suite described in §6

# 3. Build and seed your local database
python database.py --seed         # entry point is yours to define
```

2.3 Running the servers locally

Each of the three servers is a standalone MCP server and should run as its own local process, for example:

```
python -m accounts_server
python -m products_server
python -m compliance_comms_server
```

Point your MCP client (a Claude Desktop config entry, or the Anthropic API's `mcp_servers` parameter) at each running process.

2.4 Running the demo and the test suite

```
python run_local_demo.py          # exercises the tool logic described in §8, no live
MCP client needed                # required to actually run, not just exist – see §6

pytest tests/
```



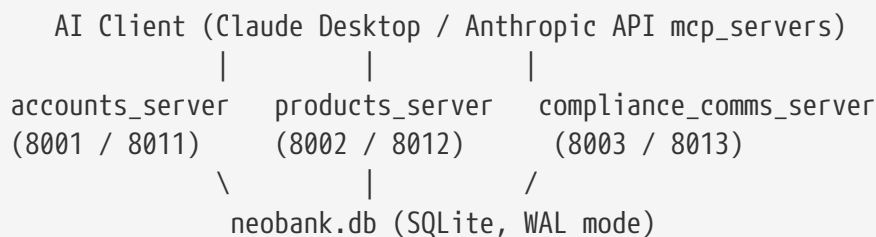
The module and entry-point names above are illustrative — build whatever structure makes sense for your implementation, and document it in your own

3. System Architecture (Requirements)

Server	Data Access	Responsibility to Implement
accounts_server	neobank.db — accounts, customers tables	get_account_summary, get_accounts_for_customer, get_transaction_history
products_server	neobank.db — loan_products table only, no customer data	list_loan_products, get_loan_product_details, check_eligibility_criteria, submit_loan_application, get_loan_application_status
compliance_comms_server	neobank.db — customers, communications_log, fraud_flags, audit_log tables	get_kyc_status, run_compliance_check, send_customer_communication, get_fraud_flags, write_audit_log, generate_customer_communication



Design each server’s data access so isolation is **actually enforced**, not incidental. `products_server` should be structurally unable to query customer data, and each server should only be able to reach the tables its own responsibility requires — this should hold whether or not you implement the three servers against one shared SQLite file or three separate ones.



3.1 Required Tool Inventory

Server	Tool	Op	Purpose
accounts_server	get_account_summary(account_id, caller_scope)	Read	Account lookup, fields minimized per caller_scope
accounts_server	get_accounts_for_customer(customer_id, caller_scope)	Read	All accounts for a customer, same minimization
accounts_server	get_transaction_history(account_id, limit)	Read	Recent transactions, most recent first
products_server	list_loan_products(category)	Read	Catalogue browse, optional category filter
products_server	get_loan_product_details(product_id)	Read	Single product lookup

Server	Tool	Op	Purpose
products_server	<code>check_eligibility_criteria(product_id, applicant_risk_rating)</code>	Read	Criteria check only — no write, no customer data
products_server	<code>submit_loan_application(...)</code>	Write	Must enforce the KYC gate at the Tool level, not leave it to the AI client
products_server	<code>get_loan_application_statuses(...)</code>	Read	Read operation over stored loan application records
compliance_comms_server	<code>get_kyc_status(customer_id)</code>	Read	Only tool anywhere returning raw KYC data
compliance_comms_server	<code>run_compliance_check(customer_id, transaction_amount)</code>	Read	Deterministic pass/block + large-transaction reporting flag
compliance_comms_server	<code>send_customer_communication(customer_id, channel, message)</code>	Write	Sanitises input, enforces KYC-based comms rule, logs the send
compliance_comms_server	<code>get_fraud_flags(...)</code>	Read	Read operation over a <code>fraud_flags</code> table
compliance_comms_server	<code>write_audit_log(...)</code>	Write	Needs its own <code>audit_log</code> table (timestamp, action type, performed-by, customer ID, outcome)
compliance_comms_server	<code>generate_customer_communication(...)</code>	Read	Must render from filesystem Markdown templates (<code>communication_templates/</code>), not a plain string



14 Tools total across the three servers, three of them write-capable (`submit_loan_application`, `send_customer_communication`, `write_audit_log`). Build `submit_loan_application` early — the KYC gate requirement (§4) and the audit-log requirement (§4) both depend on a write path that doesn't exist until this tool does. Loan applications cannot be created at all without it; only eligibility can be **checked**.



Resources and Prompts are separate required primitives — see §7 for the full inventory. They are not substitutes for any of the 14 Tools above.

4. Security & Guardrails (Requirements)

Control	Requirement
Scoped field visibility	Implement <code>guardrails.minimize_account_fields()</code> , taking an explicit <code>caller_scope</code> argument (<code>teller</code> / <code>loan_officer</code> / <code>compliance_officer</code> / <code>admin</code>) on every <code>accounts_server</code> tool call and filtering returned fields accordingly. Scoping must also govern which server can reach which data source at all — not just field filtering within one database.
Account number masking	Mask account numbers to the last four digits on every Tool or Resource that returns account data. This is a hard requirement.
Input format validation	Validate <code>CUS-XXXXX</code> / <code>ACC-XXXXX</code> / <code>PROD-XX-XX</code> ID formats on every tool/resource argument that takes one, with typed errors on mismatch.
KYC gate on write operations	Implement a gate (e.g. <code>can_send_communication()</code>) that blocks marketing messages to non-verified-KYC customers before <code>send_customer_communication</code> proceeds, and an equivalent gate before <code>submit_loan_application</code> proceeds.
Audit logging on writes	Add a dedicated <code>audit_log</code> table and a <code>write_audit_log</code> tool that produces an entry (timestamp, action type, customer ID, outcome) for every write operation, retrievable for compliance review.
Input sanitisation / prompt-injection defence	Implement <code>guardrails.sanitize_free_text()</code> — length limit, HTML stripping, regex-based injection-signature detection — and apply it to every outbound message body and any other free-text tool input.
PII redaction in logs	Implement <code>guardrails.redact_for_logging()</code> to mask PII-shaped argument names (<code>account_id</code> , <code>customer_id</code> , <code>phone</code> , <code>email</code>) before they reach a structured log line, and document how this supports data-minimisation.

5. Logging & Observability (Requirements)

Every function you write must be decorated with `@trace(logger)` in a `logging_config.py` module you build, producing:

- an **ENTER** line (DEBUG) — the function's fully qualified name, a `call_id`, and its bound arguments (redacted if a `redact=` function was supplied)
- an **EXIT** line (DEBUG) — same `call_id`, a `duration_ms`, and a truncated preview of the return value
- on exception, a **FAILED** line (ERROR) — same `call_id`, `duration_ms`, exception type, and full traceback, then the exception re-raised

All log lines must be structured JSON, one object per line. `LOG_LEVEL` should default to `DEBUG` everywhere via an environment variable.



Apply this pattern consistently across every Tool, Resource, and Prompt you build — inconsistent tracing coverage will be treated as incomplete observability, not a minor gap.

6. Testing & Verification (Requirements)

Component	Requirement
<code>logging_config.py</code> , <code>guardrails.py</code> , <code>database.py</code>	Write unit coverage and run it directly; inspect output line by line before submission
All 14 Tools' business logic	Exercise every tool function — a test-only stub of the <code>mcp</code> package standing in for <code>FastMCP</code> is acceptable during development, but you must also verify against the real package before submission
<code>pytest</code> test suite (<code>tests/</code>)	Must be written and actually executed — a written-but-unrun suite does not satisfy this requirement
Real <code>@mcp.tool()</code> / <code>FastMCP</code> registration	Verify tool/resource/prompt registration against the real <code>mcp</code> package, not only against a stub, before submission
Local server startup (each of the 3 servers as a standalone local process)	Verify each server actually starts and responds to a live MCP client — this is a submission requirement, not optional
Official Showcase Scenario (Priya Sharma / <code>CUS-10042</code> / <code>PROD-PL-01</code>)	Build seed data and fixtures for this exact scenario and demonstrate it end-to-end (see §10)
Official Stress Tests (missing customer, invalid loan submission, compliance server offline)	Build fixtures for all three and demonstrate correct handling of each (see §10)

7. MCP Primitives — Resources & Prompts (Requirements)

No MCP Resources or Prompts are provided; both must be built from scratch as required deliverables — the same as every Tool in §3.1.

Primitive	Detail
All 4 Resources	<code>customer://{customer_id}/profile</code> , <code>account://{account_id}/summary</code> , <code>product://{product_id}/details</code> , <code>template://{template_name}</code> — templated URIs, no bulk data, no unmasked sensitive fields
Both Prompts	<code>transaction_analysis_prompt</code> (args: <code>customer_name</code> , <code>account_type</code> , <code>analysis_period</code>), <code>customer_communication_prompt</code> (args: <code>template_name</code> , <code>customer_name</code> , <code>account_tier</code> , <code>specific_detail</code>) — each needs 3+ dynamic arguments



Both primitive types are required deliverables, not optional extensions. Reuse the guardrail functions from §4 inside your Resources (e.g. mask account numbers in `account://{account_id}/summary`) rather than duplicating that logic.

8. Demo Script Requirements

Build a local demo script (e.g. `run_local_demo.py`) that exercises, in order:

- scoped field visibility on the same account under at least two different `caller_scope` values
- transaction history retrieval
- loan product listing and eligibility checking
- the large-transaction reporting flag
- a KYC-blocked transaction
- a KYC-blocked marketing communication
- a blocked prompt-injection attempt
- the full Showcase Scenario (Priya Sharma / `CUS-10042` / `PROD-PL-01`)
- all three Stress Tests (missing customer, invalid loan submission, compliance server offline)

This script should run without requiring a live MCP client, so it can be executed quickly during grading.

9. Evaluation Rubric

Component	Weightage	What Assessors Look For
Working system — all required MCP primitives implemented	25%	Do all 14 required Tools run correctly? Are all 4 Resources and both Prompts (§7) implemented, reachable, and returning correct data?
Server architecture and data-source isolation	15%	Are the three servers correctly separated by responsibility? Is data-source access actually enforced per server (§3's NOTE), not merely incidental to what each tool happens to query?
Security and guardrails correctness	20%	Is account-number masking applied everywhere required? Is <code>CUS-XXXXX/ACC-XXXXX/PROD-XX-XX</code> format validation enforced with typed errors? Is the KYC gate correctly placed on <code>submit_loan_application</code> ? Does <code>write_audit_log</code> produce a complete, retrievable entry for every write operation?
MCP primitive design quality (Tools vs. Resources vs. Prompts)	15%	Are Tools, Resources, and Prompts used for what they're each meant for — actions via Tools, data via Resources, reusable instructions via Prompts? Do the Resources reuse existing guardrail functions rather than duplicating logic? Do the Prompts direct the client to the right Tool/Resource calls instead of embedding sensitive data directly?
Logging, observability, and testing	10%	Is every Tool, Resource, and Prompt traced via <code>@trace(logger)</code> consistently? Was the <code>pytest</code> suite actually installed and run, with any failures triaged and fixed?
Demo coverage — Showcase Scenario and Stress Tests	10%	Does the demo script actually exercise the Showcase Scenario (Priya Sharma / <code>CUS-10042</code> / <code>PROD-PL-01</code>) and all three Stress Tests end-to-end?
Architecture documentation and presentation	5%	Is the "what did not work" section honest and diagnostic? Can every team member explain every design decision under panel questioning?